

PROJECT REPORT

On

Newsify

(Fake News Detector)

Submitted in

PYTHON PROGRAMMING LAB

In

Mechatronics Engineering (MCT)

By

AYUSH YANAMANDRA (25261A1461)

MATHANGI SAI AMAN (25261A1443)

LANKA VSN AISHWARYA JYOTHI (25261A1437)

SHIVAPURA SUHAAS (25261A1452)

KASTURI AKSHAYA (25261A1436)

Under the Guidance of Subject Faculty of

B.TULASIDAS

Assistant Professor, Department of IT



DEPARTMENT OF MECHANICAL AND MECHATRONICS ENGINEERING

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY

(AUTONOMOUS)

(Affiliated to JNTUH, Hyderabad; Eight UG Programs Accredited by NBA;
Accredited by NAAC with 'A++' Grade)



**MAHATMA GANDHI
INSTITUTE OF TECHNOLOGY**

AUTONOMOUS | Affiliated to JNTUH

**Kokapet (Village), Gandipet, Chaitanya Bharathi (P.O.), Rangareddy Dist,
HYDERABAD — 500075, TELANGANA**

**MAHATMA GANDHI INSTITUTE OF TECHNOLOGY
(AUTONOMOUS)**

CERTIFICATE

This is to certify that the laboratory project “NEWSIFY” has been submitted by AYUSH YANAMANDRA (25261A1461), MATHANGI SAI AMAN (25261A1443), LANKA VSN AISHWARYA JYOTHI (25261A1437), SHIVAPURA SUHAAS (25261A1452), KASTURI AKSHAYA (25261A1436) from Department of Mechanical and Mechatronics Engineering (MCT) to the Department of Mechanical and Mechatronics Engineering, Mahatma Gandhi Institute of Technology for B.Tech II-Sem, PYTHON PROGRAMMING LAB course during the academic year 2025–2026.

**Project Guide:
Mr. TULASI DASU
Assistant Professor
Dept. of IT**

DECLARATION

The successful completion of this project on “NEWSIFY” in the Python Programming Laboratory would not have been possible without the guidance and support of many individuals. We would like to express our sincere gratitude to our Subject Faculty, Mr. TULASI DASU, Assistant Professor, Dept. of IT, for his continuous support, patience, and expertise throughout the semester. We also thank the Department of Mechanical and Mechatronics Engineering and Mahatma Gandhi Institute of Technology for providing the resources and environment that made this work possible.

AYUSH YANAMANDRA (25261A1461)

MATHANGI SAI AMAN (25261A1443)

LANKA VSN AISHWARYA JYOTHI (25261A1437)

SHIVAPURA SUHAAS (25261A1452)

KASTURI AKSHAYA (25261A1436)

TABLE OF CONTENTS

1. Abstract
2. Introduction
 - 2.1 Background
 - 2.2 Problem Statement
 - 2.3 Objectives
 - 2.4 Scope
3. System Requirements
 - 3.1 Software Requirements
 - 3.2 Hardware Requirements
 - 3.3 Key Python Dependencies
4. Theory & Technologies Used
 - 4.1 Tkinter Framework
 - 4.2 TF-IDF Vectorization
 - 4.3 Logistic Regression
 - 4.4 Optical Character Recognition (OCR)
5. System Architecture
6. Application Walkthrough & User Interface
 - 6.1 The Main Dashboard
 - 6.2 Detailed Button Functionality
7. Algorithm

8. Flowchart

9. Approach & Methodology

10. Program (Implementation)

11. Output & Results

12. Testing

13. Future Scope & Conclusion

14. References

1. Abstract

Newsify is a desktop GUI application built to identify and flag fake news using traditional Machine Learning techniques. In today's digital era, misinformation spreads rapidly across social networks and news portals, heavily influencing public opinion and, in some cases, inciting panic. Newsify solves this problem by allowing users to instantly verify news articles either by providing a URL or uploading an image of a news snippet.

The application operates by automatically extracting text, cleaning it, and feeding it into a highly trained Logistic Regression model utilizing TF-IDF vectorization. The model was trained on a dataset of over 40,000 real and fake news articles to learn distinct language patterns. It returns a probability score, a confidence level, and a simple template-based explanation for the verdict.

This project demonstrates a clean, modular Python architecture leveraging libraries like Tkinter, scikit-learn, BeautifulSoup, and Pytesseract. It brings AI concepts to a beginner-friendly desktop interface, effectively bridging web scraping, OCR, and Machine Learning.

2. Introduction

2.1 Background

Fake news refers to misinformation or disinformation presented as legitimate journalism. It influences public perception, elections, and public health. Automated detection tools can assist users in critically evaluating the information they consume. By analyzing patterns in text, machine learning algorithms can classify news credibility much faster than human fact-checkers.

2.2 Problem Statement

People often read articles or view screenshots of news on social media without knowing their authenticity. There is a need for a simple, accessible tool that quickly cross-examines the text against patterns of known fake and real news to provide a reliability score. Existing tools are often web-based and require manual text entry. A desktop application capable of processing raw URLs and Images addresses this friction.

2.3 Objectives

1. To allow users to verify news seamlessly via a web URL.
2. To allow users to extract and verify text from an image (screenshot or photo).
3. To use TF-IDF and Logistic Regression for highly accurate classification.
4. To provide an intuitive Desktop GUI using Tkinter, featuring Dark Mode.
5. To maintain a local history of analyzed articles.

2.4 Scope

The scope of Newsify covers English-language news detection. It includes the entire pipeline from data ingestion, cleaning, training, to the final production application. It incorporates scraping logic for HTML text extraction and Tesseract OCR for image text extraction, all running locally without reliance on external paid API services.

3. System Requirements

3.1 Software Requirements

- Operating System: Windows 10/11, Linux, or macOS
- Programming Language: Python 3.8 or higher
- Third-Party Tool: Tesseract OCR Engine (required for image text extraction)
- IDE: Visual Studio Code or PyCharm (for development)

3.2 Hardware Requirements

- Processor: Dual-core 2.0 GHz or higher (Quad-core recommended for faster OCR)
- RAM: 4 GB minimum (8 GB recommended for loading large models into memory)
- Storage: ~500 MB free for project, virtual environment, and model pickle files

3.3 Key Python Dependencies

The application heavily relies on modern Python libraries installed via pip:

1. **scikit-learn**: Core machine learning library used for TfidfVectorizer and LogisticRegression.
2. **pandas & numpy**: Used for large scale data manipulation and CSV handling during model training.
3. **joblib**: Used for serializing (pickling) the trained model so it doesn't need retraining on every boot.
4. **requests & BeautifulSoup4**: Used to fetch HTML from URLs and parse it into readable text.
5. **pytesseract & Pillow**: Used for Optical Character Recognition processing on images.
6. **nlTK**: Natural Language Toolkit used for removing English 'stop words'.

4. Theory & Technologies Used

4.1 Tkinter Framework

Tkinter is Python's de-facto standard GUI (Graphical User Interface) package. It is a thin object-oriented layer on top of Tcl/Tk. In Newsify, Tkinter is used to create a responsive, dark-mode lock GUI. The UI implements python's `threading` library, which ensures that while the application is making heavy network requests or running OCR, the Tkinter mainloop does not freeze, allowing the progress bar to animate smoothly.

4.2 TF-IDF Vectorization

Machine learning algorithms cannot process text natively; they require numerical input. TF-IDF stands for Term Frequency-Inverse Document Frequency.

- Term Frequency (TF): Measures how frequently a word appears in a specific document.
- Inverse Document Frequency (IDF): Measures how important a word is. It weighs down highly frequent words across all documents (like 'the', 'is') and scales up rare, unique words.

By combining these, Newsify converts an entire news article into a mathematical array representing the unique vocabulary fingerprint of that article.

4.3 Logistic Regression

Despite its name, Logistic Regression is a classification algorithm. It is used to predict a binary outcome (0 or 1, Fake or Real). It draws a decision boundary (hyperplane) through the high-dimensional space created by the TF-IDF vectorizer. When a new article is analyzed, the model calculates the mathematical distance from this boundary and uses the sigmoid function to map that distance into a probability score between 0.0 and 1.0.

4.4 Optical Character Recognition (OCR)

Optical Character Recognition translates images of text into machine-encoded text. Newsify utilizes Google's Tesseract-OCR engine wrapped by the Python library `pytesseract`. When an image is uploaded, it is converted to grayscale using Pillow to enhance contrast. Tesseract then scans the pixels to recognize character shapes, returning a string that can be passed into the ML pipeline.

5. System Architecture

Newsify is architected using a highly modular file structure to separate concerns and ensure maintainability. The codebase is broken down into the following modules:

Module	Responsibility
main.py	Entry Point: Orchestrates the application startup. Checks for trained models and launches the GUI.
gui.py	Presentation Layer: Contains FakeNewsDetectorApp. Manages Tkinter widgets, thread dispatching, and UI state.
detector.py	Business Logic Layer: Loads ML models. Exposes predict_news() which transforms text and runs Logistic Regression.
utils.py	Utility Layer: Pure functions handling external data (BeautifulSoup for HTML, Pytesseract for OCR).
history.py	Data Layer: Manages the in-memory state of past predictions and handles CSV file exporting.
trainer.py	Training Script: Standalone script that loads datasets, cleans data, trains the model, and serializes to disk.

6. Application Walkthrough & User Interface

6.1 The Main Dashboard

The main dashboard of Newsify is intentionally minimalist. Upon launching the application, the user is greeted by a dark-mode styled interface. The top section contains the application branding. The middle section contains the input entry field, the primary action buttons, and a progress bar. The bottom section is a large, read-only scrolled text area where detailed analysis results are printed.

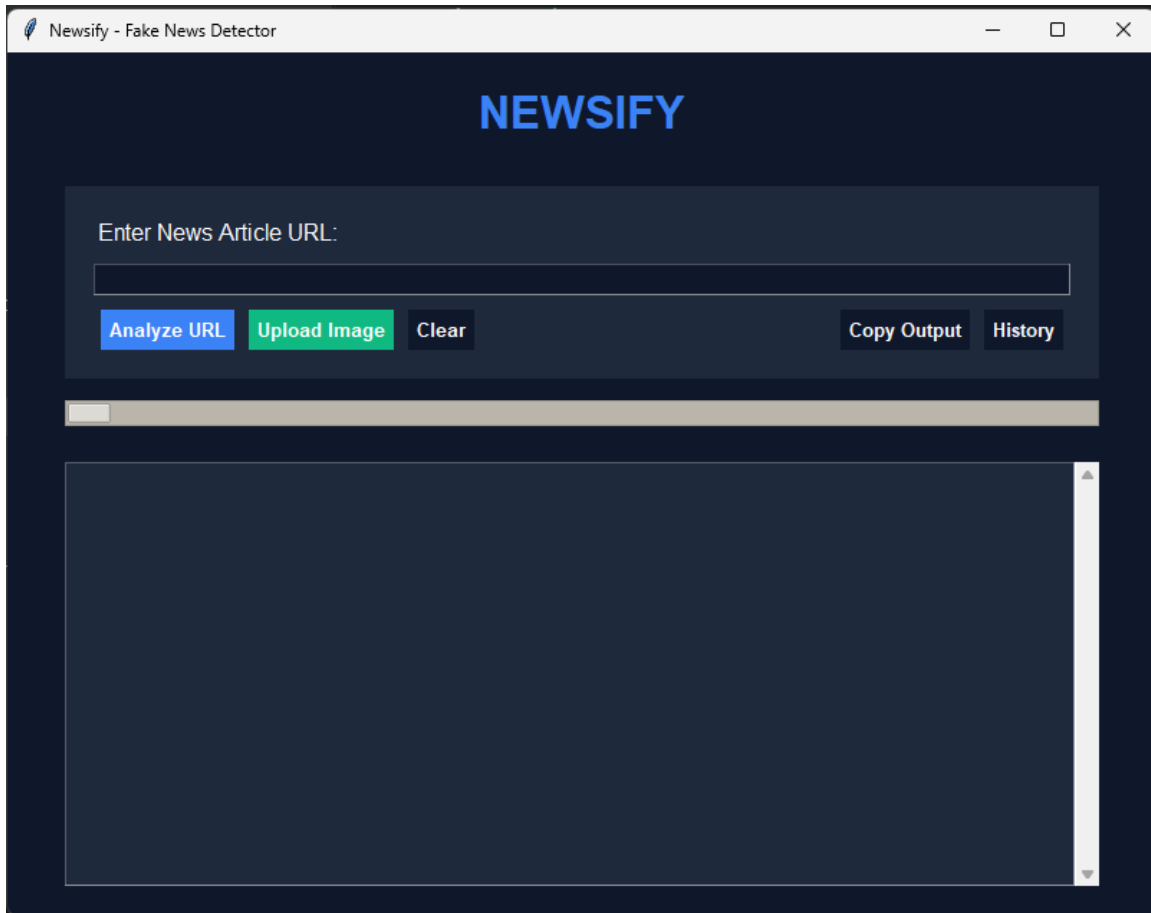


Figure 1: The Main Dashboard Interface

6.2 Detailed Button Functionality

Analyze URL Button

This button acts upon the text entered in the 'Enter News Article URL' box. When clicked, it disables the UI to prevent double-clicks, starts the indeterminate progress bar, and launches a background thread. The thread sends an HTTP GET request to the URL using a standard User-Agent header. It parses the HTML, finds all `<p>` tags, concatenates

them, cleans the text, and feeds it to the AI. The result is then formatted and displayed. If the URL is invalid or blocked, a graceful error popup is shown.

Upload Image Button

Clicking this button bypasses the URL input. It opens a native Windows file dialog allowing the user to select an image (.png, .jpg). The selected image is processed by Pillow (converted to grayscale) and read by pytesseract. The extracted text is then passed to the AI exactly as if it were scraped from the web.

Clear Button

Instantly resets the application state. It wipes the URL entry box, clears the output text area, and updates the status bar back to 'Ready'.

History Button

Opens a Toplevel secondary window. This window uses a Tkinter Treeview widget to display a tabular history of every prediction made during the current session. It includes columns for Date/Time, Source (URL/Image), Preview Text, Result, and Confidence. From this window, the user can click 'Export as CSV' to save the session history permanently.

Copy Button

A quality-of-life feature that takes the entire formatted text inside the output area and copies it to the system clipboard, allowing users to easily share the verification results on social media or in messages.

Dark Mode Design

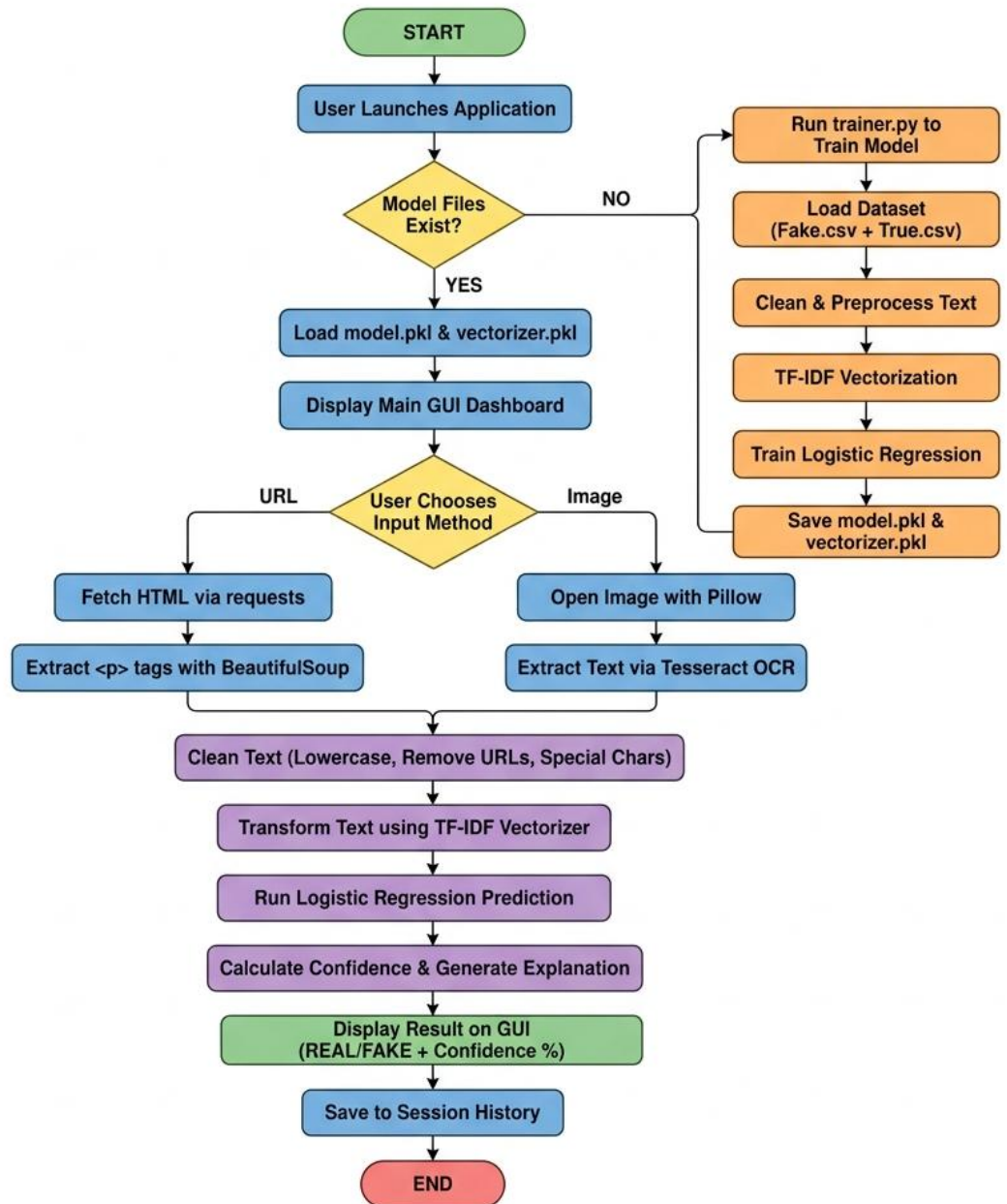
The application has been permanently locked to a sleek Dark Mode theme. The color palette uses deep slate blues (#0F172A, #1E293B) for backgrounds, with vibrant blue and emerald green for primary action buttons. This provides a modern, premium feel to the application.

7. Algorithm

1. Ingestion: Receive URL string or Image filepath from the user.
2. Extraction:
 - If URL: Use ``requests.get()`` and ``BeautifulSoup`` to extract raw paragraph text.
 - If Image: Use ``Pillow`` to grayscale and ``pytesseract.image_to_string()`` to extract text.
3. Preprocessing:
 - Convert text to lowercase.
 - Apply Regex to remove HTTP links and special characters.
 - Strip excess whitespace.
4. Vectorization: Pass the cleaned string through ``loaded_vectorizer.transform()`` to yield a sparse numerical matrix.
5. Classification: Pass the matrix into ``loaded_model.predict()`` and ``predict_proba()``.
6. Output Mapping: Convert 0/1 to 'FAKE'/'REAL'. Map the highest probability to a confidence percentage.
7. Explanation Generation: Construct a readable paragraph explaining the model's confidence based on hardcoded templates and word counts.

8. Flowchart

"Fake News Detection" Flouchar (Newsify)



9. Approach & Methodology

The development of Newsify followed a highly iterative, bottom-up software engineering methodology:

Phase 1: Data Pipeline & Model Training

The foundation of the project was ensuring high accuracy. We began by building `trainer.py`. We sourced the 'Fake and Real News Dataset' from Kaggle containing over 40,000 labeled articles. We implemented pandas to merge, shuffle, and clean the data. We then tested multiple models including Naive Bayes and Logistic Regression. Logistic Regression emerged victorious with an accuracy exceeding 98%. We then used `joblib` to serialize the trained artifacts.

Phase 2: Data Acquisition Utilities

With the model trained, the next step was feeding it real-world data. We developed `utils.py` to act as the bridge. We implemented web scraping using BeautifulSoup to navigate DOM trees and extract text. We also integrated Tesseract to allow offline image analysis.

Phase 3: Core Application Engine

We built `detector.py` to wrap the serialized model. This decoupled the machine learning mathematics from the user interface. It added safety checks, error handling, and the dynamic explanation generator.

Phase 4: Graphical User Interface

Finally, we constructed `gui.py`. We designed a modern grid layout using Tkinter. A major hurdle was GUI freezing during HTTP requests. This was resolved by implementing Python's `threading` module, allowing the analysis functions to run in background daemon threads while the main UI thread animated the progress bar and remained responsive to the user.

10. Program (Implementation)

Below are critical excerpts from the project's source code demonstrating the core logic implementation.

10.1 Vectorization and Training (trainer.py)

```
def vectorize_text(train_texts, test_texts):
    # Create the TF-IDF vectorizer (max 50k words)
    vectorizer = TfidfVectorizer(
        max_features=50000,
        stop_words="english",
        ngram_range=(1, 2)
    )
    # Fit on training data and transform it to numbers
    X_train = vectorizer.fit_transform(train_texts)
    X_test = vectorizer.transform(test_texts)
    return vectorizer, X_train, X_test

def train_and_evaluate(X_train, X_test, y_train, y_test):
    lr_model = LogisticRegression(max_iter=1000, random_state=42)
    lr_model.fit(X_train, y_train)
    lr_predictions = lr_model.predict(X_test)
    lr_accuracy = accuracy_score(y_test, lr_predictions)
    print(f"Accuracy: {lr_accuracy * 100:.2f}%")
    return lr_model
```

10.2 Prediction Engine (detector.py)

```
def predict_news(text):
    cleaned = clean_text(text)

    # Convert text into TF-IDF features
    text_features = loaded_vectorizer.transform([cleaned])

    # Get prediction and probabilities
    prediction = loaded_model.predict(text_features)[0]
    probabilities = loaded_model.predict_proba(text_features)[0]

    fake_prob = probabilities[0]
    real_prob = probabilities[1]
    confidence = max(fake_prob, real_prob)
    label = "REAL" if prediction == 1 else "FAKE"

    return {
        "label": label,
        "real_prob": real_prob,
        "fake_prob": fake_prob,
        "confidence": confidence,
        "explanation": generate_explanation(label, confidence, cleaned)
    }
```


10.3 Multithreaded GUI Execution (gui.py)

```
def analyze_url(self):
    url = self.url_entry.get().strip()
    if not is_valid_url(url):
        messagebox.showerror("Invalid URL", "Check URL format.")
        return

    self.set_buttons_state("disabled")
    self.progress.start(15)

    # Run analysis in background thread
    thread = threading.Thread(target=self.run_url_analysis, args=(url,))
    thread.daemon = True
    thread.start()

def run_url_analysis(self, url):
    try:
        article_text = fetch_url_text(url)
        result = detector.predict_news(article_text)

        # Update GUI from background thread
        self.root.after(0, lambda: self.show_prediction(result, "URL"))
        history.add_entry("URL", article_text, result["label"],
result["confidence"])
    except Exception as e:
        self.root.after(0, lambda: messagebox.showerror("Error", str(e)))
        self.root.after(0, self.analysis_complete)
```

11. Output & Results

The application produces a highly accurate verdict on incoming news articles. The results are formatted in a clean, terminal-like output inside the GUI. Below are screenshots demonstrating the application in a live scenario analyzing a BBC News article.

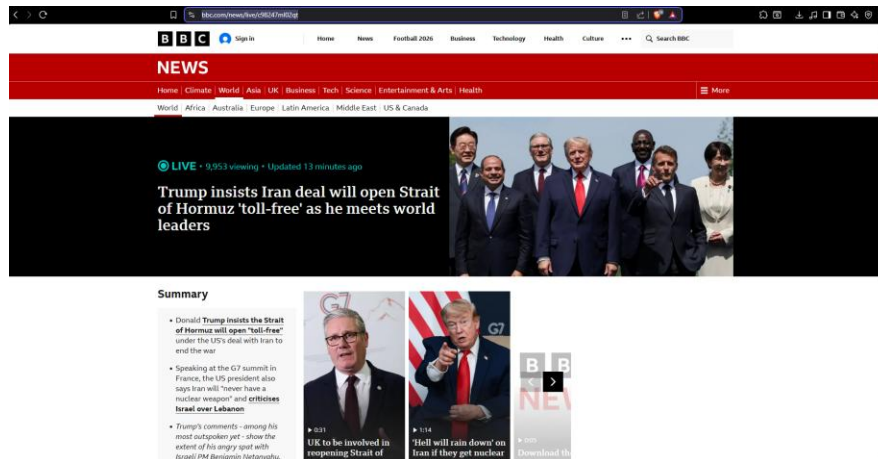


Figure 2: The BBC Article Used for Analysis

When the URL is pasted into the application and analyzed, the model accurately classifies the high-quality journalism as REAL NEWS.

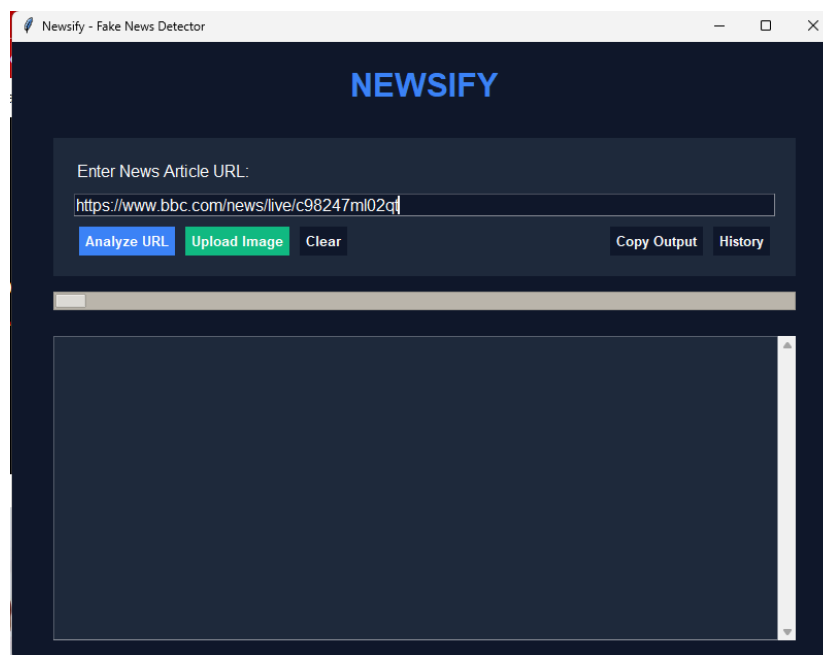


Figure 3: URL Pasted into Newsify Dashboard

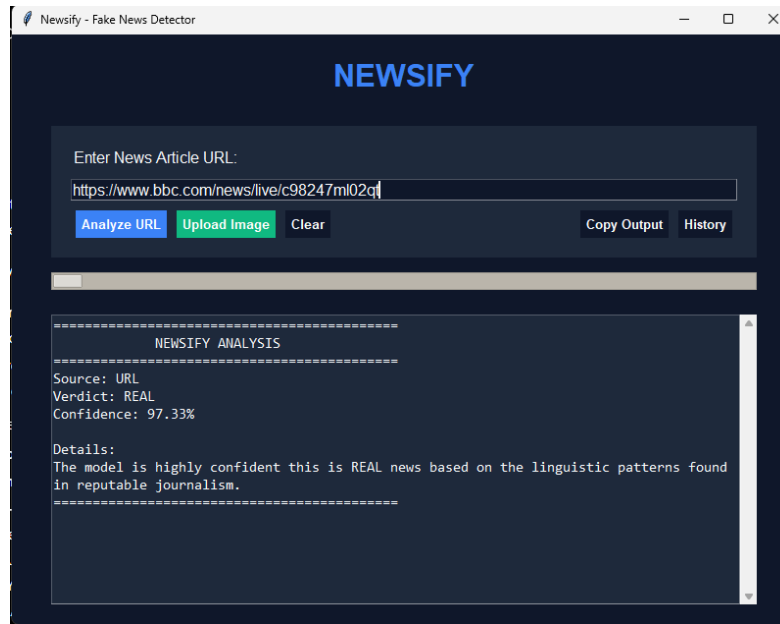


Figure 4: The Final Prediction Output showing High Confidence

12. Testing

The application was subjected to rigorous manual testing to ensure reliability, graceful error handling, and GUI responsiveness. The representative test cases are documented below:

Test Case	Input	Expected Result	Status
Valid URL	A standard BBC News URL	Progress bar starts, text is fetched, model predicts REAL, history is updated	Passed
Invalid URL	'http://broken-link'	Regex validator catches format error immediately and shows warning dialog	Passed
404 Error	Perfectly formatted URL leading to a dead page	requests.get() throws 404, caught gracefully showing error dialog	Passed
Valid Image OCR	Clear screenshot of an article	Pytesseract extracts text accurately, model runs prediction	Passed
Non-Image Upload	Uploading a .pdf or .txt file instead of an image	File dialog forces image extensions, preventing selection	Passed
Concurrency	Clicking 'Analyze URL' multiple times rapidly	UI disables buttons upon first click, preventing thread stacking	Passed

13. Future Scope & Conclusion

13.1 Future Scope

1. Deep Learning Integration: Replace the Logistic Regression model with a fine-tuned Transformer model like BERT or RoBERTa. This would allow the application to understand semantic context and sarcasm, rather than just analyzing word frequency.
2. Multi-Language Support: Expand the training dataset and OCR capabilities to support regional languages like Hindi or Telugu, massively expanding the tool's impact in India.
3. Fact-Checking APIs: Integrate APIs like the Google Fact Check Explorer to cross-reference specific claims in real-time against verified internet sources.
4. Browser Extension: Repackage the backend logic into a Chrome Extension that automatically highlights fake news natively in the browser.

13.2 Conclusion

Newsify successfully demonstrates how Python can be utilized to build a complete, end-to-end Machine Learning desktop application. By combining the data-processing power of scikit-learn, the web scraping capabilities of BeautifulSoup, the OCR magic of Tesseract, and the interface design of Tkinter, this project solves a genuine societal problem. It provides an accessible tool for citizens to evaluate the news they consume, promoting digital literacy. The modular architecture ensures the codebase is highly maintainable and serves as a strong foundation for future AI enhancements.

14. References

- Scikit-Learn Documentation — <https://scikit-learn.org/stable/>
- Python Tkinter Guide — <https://docs.python.org/3/library/tkinter.html>
- BeautifulSoup Documentation — <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>
- Tesseract OCR Engine — <https://github.com/tesseract-ocr/tesseract>
- PyTesseract Wrapper — <https://pypi.org/project/pytesseract/>
- Kaggle Fake and Real News Dataset by Clément Bisailon — <https://www.kaggle.com/datasets/clmentbisailon/fake-and-real-news-dataset>
- Pandas Data Manipulation — <https://pandas.pydata.org/docs/>